
```

%Part 1: data preparation folder, file, dataset
folder =
["C:\Users\khang\OneDrive\Documents\MATLAB\PROJECT\Data\Male","C:\Users\khang\OneDrive\Documents\MATLAB\PROJECT\Data\Female"];
folderlabel= ["male", "female"];

%part2: set all record as same frequency speed
fsSet = 16000; % all record need to be consistency
labels = []; % empty label vector for male and female
features = []; % empty features vector

for k = 1 : length(folder)
    fol = folder(k);
    label = folderlabel(k);
    files = dir(fullfile(fol, '*.wav'));
    % Load audio files from the folder

    for i = 1:length(files)
        fullpath = fullfile(fol, files(i).name); %build full file path to
        linking record in folder
        [y,Fs]= audioread(fullpath); %audio read file

        % No trim because record only 1,2s, trim make it too short and pitch
        % cannot run

        % Monoaural - Mono audio has only 1 channel
        % convert mono for simpler processing, and consistent dataset for all
        % features.
        if size(y,2) > 1
            y = mean(y,2);
        end %mono function
        if ~isempty(y) % Normalize the audio signal
            y = y / max(abs(y));
        end
        % normalize - finds the loudest point in the audio
        % same reason of mono, consistency dataset, improve data integrity for
        feature extraction
        if Fs ~= fsSet
            y=resample(y,fsSet, Fs);
            Fs = fsSet;
        end
        % Set same sample rate
        % change the sample rate to value above
        % consistency and easier and faster processing to correct feature

    end

%pitch features - fundamental frequency F0
%Pitch is the perceptual property of sounds. A higher frequency of vocal
fold vibration results in a higher perceived pitch.
% Male 85 to 155 Hz, female 165 to 255 Hz

```

```

win = round(fsSet * 0.03);
ov = round(0.015*fsSet);
p = pitch(y, fsSet, 'range', [50 350],
'WindowLength', win, 'OverlapLength', ov);

% calculates fundamental frequency of record

pMean=median(p);% The average of pitch of record
% Male 85 to 155 Hz, female 165 to 255 Hz
pStd=std(p);
pIQR=iqr(p);
%The Interquartile Range (IQR) is a statistical measure of the variable
in mid 50% of a set of pitch data
% we use IQR to measures the variable the pitch is across the recording

%MFCC features: MFCC is a timbre of the voice
% Mel - Frequency Ceptral coefficients
numCoeffs = 13;
mf = mfcc(y, fsSet, 'NumCoeffs', numCoeffs); % function to calculate MFCC
% vector contains the values for a small frame of the audio
mfMean=median(mf,1);
% same as pitch, we want a mean of MFCC across all frame as
% characteristics of voice in the record
mfSTD=std(mf,1);
% show the standard or variability of voice over time

%spectral features include 2 feature
% we want to know the center of mass of the spectrum - centroid
% Bandwidth as the the range of frequencies
centroid = spectralCentroid(y,fsSet);
bandwidth =spectralSpread(y, fsSet);
% computes the spectral centroid and bandwidth and the output are vector,
% one value per frame
centroid_mean = mean(centroid);
centroid_std = std(centroid);

bandwidth_mean = mean(bandwidth);
bandwidth_std = std(bandwidth);
rolloff = spectralRolloffPoint(y, fsSet);
rolloff_mean = mean(rolloff);
% basically rolloff is feature measure the frequency below which a specific
percentage of the total energy is contained
% all we looking for the average, from the machine can learning to classify

rmsVal = rms(y);
zcr = sum(abs(diff(sign(y))))/length(y);

feat = [rmsVal,pMean, pIQR,pStd, mfMean, mfSTD, centroid_mean,
centroid_std,bandwidth_std, bandwidth_mean, rolloff_mean, zcr];
features=[features; feat];
labels = [labels; label];

```

```

end
end
y =categorical(labels);
train = cvpartition(y, 'Holdout', 0.3) ;
% Prepare training and testing datasets
XtrainData = features(training(train), :);
YtrainLabels = y(training(train));
XtestData = features(test(train), :);
YtestLabels = y(test(train));
fprintf("Training samples : %d\n", length(YtrainLabels));
fprintf("Testing samples : %d\n", length(YtestLabels));
%train classifier

k = 5;
Mdl = fitcknn(XtrainData, YtrainLabels, "NumNeighbors",k);
% Predict using KNN model
knnPred = predict(Mdl, XtestData);
% Calculate accuracy for KNN model
knnAccur = mean(knnPred == YtestLabels);
fprintf("Test accuracy is %.2f%%\n", knnAccur* 100);

%predict by fitcsvm

model = fitcsvm(XtrainData,
YtrainLabels,'KernelFunction','rbf','Standardize',true);
pred = predict(model, XtestData);
% Accuracy
accur = mean(pred==YtestLabels);
fprintf("The Accuracy: %.2f%%\n", accur*100);

% plot the confusion matrix
figure;
confusionchart(YtestLabels, pred,"RowSummary","row-
normalized","ColumnSummary","column-normalized");
title('Gender Classification');

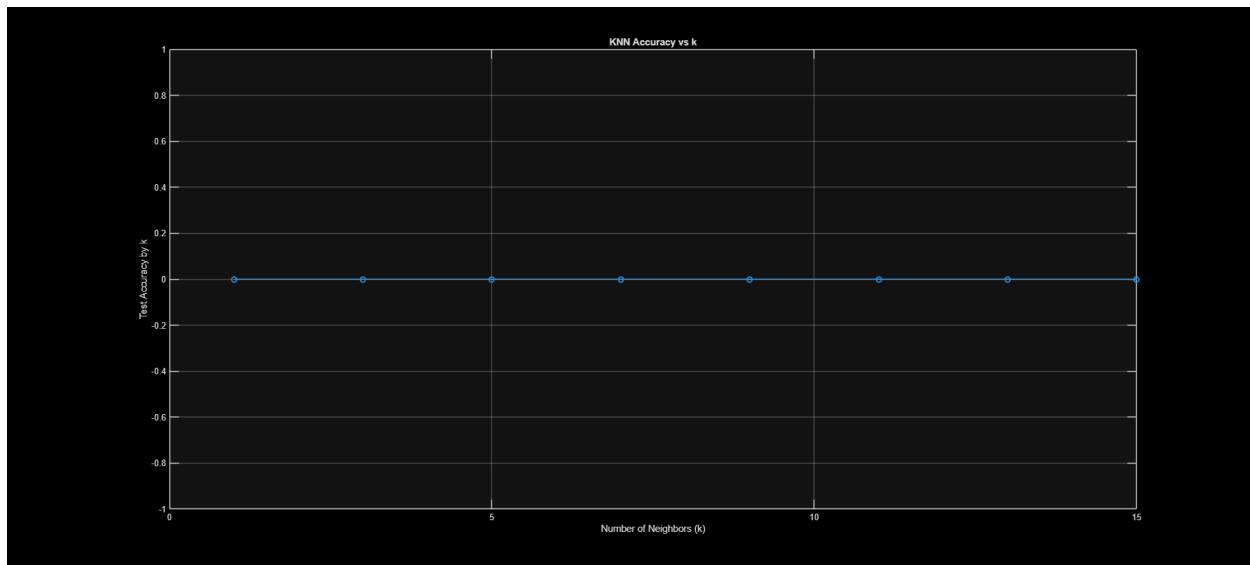
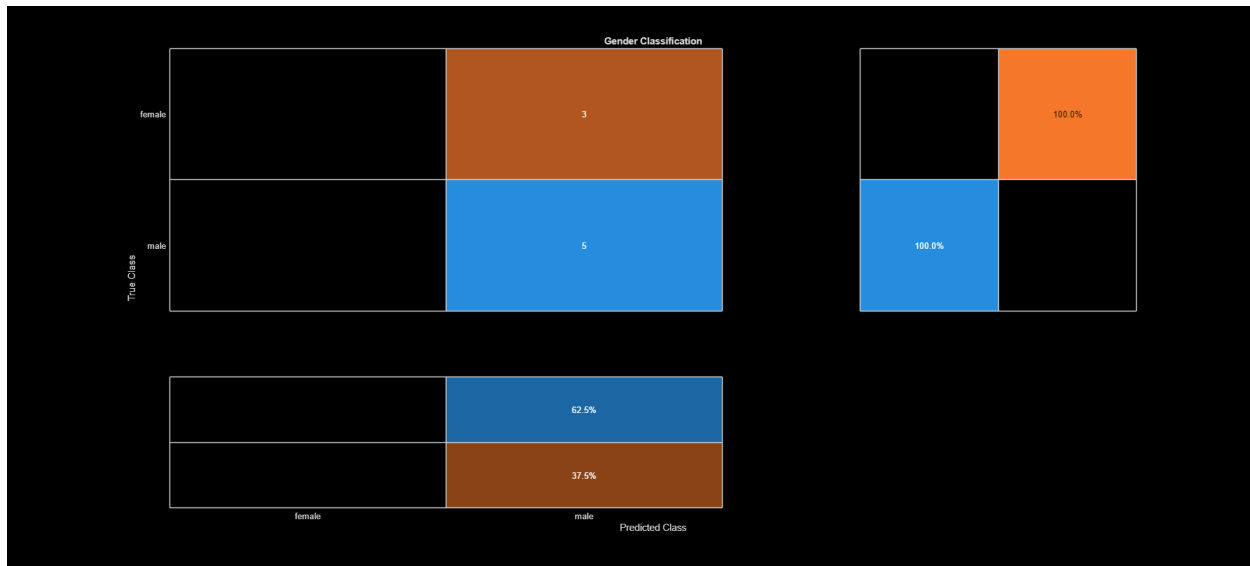
%Try different k values
k_values = 1:2:15;
acc_values = zeros(size(k_values));

for i = 1:numel(k_values)
    k_i = k_values(i);
    Mdl_i = fitcknn(XtrainData, YtrainLabels, 'NumNeighbors', k_i);
    Ypred_i = predict(Mdl_i, XtestData);
    acc_values(i) = mean(strcmp(Ypred_i, YtestLabels));
end

figure;
plot(k_values, acc_values, '-o', 'LineWidth', 1.5);
xlabel('Number of Neighbors (k)');
ylabel('Test Accuracy by k');
title('KNN Accuracy vs k');
grid on;

```

Training samples : 19
Testing samples : 8
Test accuracy is 62.50%
The Accuracy: 62.50%



Published with MATLAB® R2025a